РО 4.4 • Лекция №7

CORS и безопасность HTTP-взаимодействия

Same-Origin Policy, preflight, Express, cookie и CSRF • Internet Shop

Что такое CORS

- CORS (Cross-Origin Resource Sharing — совместное использование ресурсов между разными источниками) — это механизм безопасности браузера, основанный на HTTP-заголовках, который позволяет серверу указывать, каким веб-сайтам с другим источником (origin) разрешено читать ответы на запросы к его ресурсам.

- Пример: frontend работает на http://localhost:5500, а backend — на http://localhost:3000. Это разные источники, поэтому для доступа JavaScript к ответам API требуется соответствующее разрешение CORS.

Зачем изучать CORS

- Понимать, почему запрос из браузера может завершиться CORS-ошибкой, хотя сервер работает.
- Настраивать Express только тогда, когда frontend и API используют разные origins.
- Отличать CORS от проверки JWT и прав пользователя.
- Понимать роль cookie и CSRF без усложнения нашего учебного Internet Shop.

Учебные вопросы

№	Учебный вопрос	Что узнаем
1	Origin и Same-Origin Policy	Что браузер считает одним источником
2	CORS и предварительный запрос OPTIONS	Как браузер проверяет разрешение
3	Настройка CORS в Express	Как разрешить нужный frontend и проверить работу
4	Credentials и cookies	Чем cookie отличается от Bearer JWT
5	CSRF и безопасность	Как защитить действия пользователя

Наш Internet Shop: нужен ли CORS?

- Сейчас frontend: http://localhost:3000/; API: http://localhost:3000/api/products.
- У обоих адресов одинаковые scheme, host и port — это один origin.
- Поэтому для текущих ЛР-5 и ЛР-6 CORS не требуется.
- Для изучения CORS рассмотрим отдельный учебный вариант frontend на localhost:5500 и API на localhost:3000.

Учебный вопрос 1. Origin и Same-Origin Policy

- Сначала разберём, по какому правилу браузер отличает один источник от другого.

Определение: origin (источник)

- Origin — сочетание протокола (scheme), имени хоста (host) и порта (port).
- Пример: http://localhost:3000. Здесь http — протокол, localhost — хост, 3000 — порт.
- Путь /api/products и фрагмент #/login НЕ входят в origin.
- Различие хотя бы одной части означает другой origin.

Сравнение адресов: один origin или нет?

Страница	API	Один origin?
http://localhost:3000/	http://localhost:3000/api/products	Да
http://localhost:5500	http://localhost:3000	Нет: порт
http://shop.local	https://shop.local	Нет: протокол
https://shop.example.com	https://api.example.com	Нет: хост

Определение: Same-Origin Policy (SOP)

- Same-Origin Policy — политика браузера, ограничивающая доступ JavaScript одного origin к данным другого.
- Она препятствует произвольному чтению чужих ответов из открытой вкладки.
- SOP не означает, что любой HTTP-запрос обязательно запрещён.
- Для разрешённого чтения ответов между origins существует CORS.

Same-Origin Policy: frontend :5500 и backend :3000 — разные origins

Один и тот же сайт (scheme + host + port) — это один origin. Если отличается хотя бы одна часть — это другой origin.

Frontend

http://localhost:5500



scheme:	http
host:	localhost
port:	5500

Browser

Same-Origin Policy

HTTP request (от JavaScript)

```
fetch('http://localhost:3000/api')
```

Без CORS browser блокирует доступ JavaScript к response

Ответ от сервера приходит, но браузер не отдаёт его в JavaScript из-за нарушения Same-Origin Policy.

Backend / Express API

http://localhost:3000



Express

scheme:	http
host:	localhost
port:	3000

Origin = scheme + host + port

Frontend origin
http://localhost:5500

≠

Backend origin
http://localhost:3000

Здесь **scheme (http)** и **host (localhost)** совпадают, но **port** разный (5500 ≠ 3000), поэтому это разные origins. Без CORS браузер блокирует доступ JavaScript к response.

Что произойдёт без CORS?

- JavaScript страницы localhost:5500 запрашивает localhost:3000/api/products.
- Сервер может обработать запрос и отправить ответ.
- Если ответ не содержит подходящих разрешающих CORS-заголовков, браузер не предоставляет его вызывающему JavaScript.
- Причина обычно видна в DevTools → Console; детали запроса — в Network.

Учебный вопрос 2. CORS и preflight

- Теперь узнаем, как сервер сообщает браузеру о разрешённых источниках.

Определение: CORS

- CORS (Cross-Origin Resource Sharing) — механизм HTTP-заголовков, который позволяет серверу разрешать браузерному JavaScript другого origin читать ответы.
- Браузер может отправить заголовок Origin с адресом источника страницы.
- Сервер отвечает Access-Control-Allow-Origin с разрешённым origin.
- Решение о доступе JavaScript к ответу принимает браузер.

Пример: запрос и ответ CORS

```http
GET /api/products HTTP/1.1
Origin: http://localhost:5500

HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5500
```

Что важно

- Origin — адрес frontend, с которого отправлен запрос.
- Access-Control-Allow-Origin — указанный сервером разрешённый источник.
- Это HTTP-заголовки, а не команды JavaScript.

Определение: preflight (предварительная проверка)

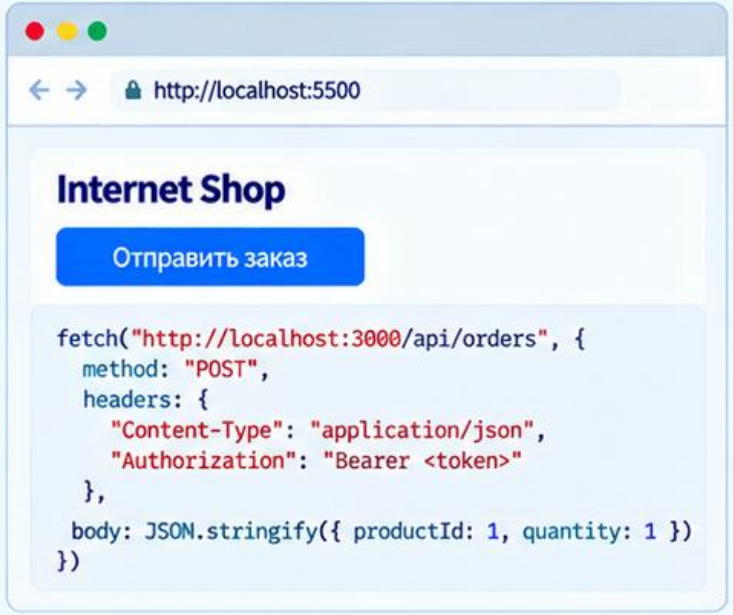
- Preflight — дополнительный запрос OPTIONS, который браузер иногда отправляет перед основным cross-origin запросом.
- Так браузер заранее уточняет, разрешены ли метод и заголовки будущего запроса.
- Preflight не нужен для всех запросов; простой GET часто обходится без него.
- Разработчик обычно не отправляет OPTIONS вручную.

CORS Preflight: OPTIONS → CORS response → Actual request

Для «непростых» запросов (POST + application/json + Authorization) браузер сначала отправляет preflight (OPTIONS), проверяет CORS headers, и только затем выполняет настоящий запрос.

Internet Shop
Learn. Build. Deploy.

Браузер (Frontend)
http://localhost:5500




```
fetch("http://localhost:3000/api/orders", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer <token>"
  },
  body: JSON.stringify({ productId: 1, quantity: 1 })
})
```

Это **не простой** запрос, так как:
- метод POST (не GET/HEAD/POST с простыми заголовками)
- Content-Type: application/json
- есть заголовок Authorization

→ Браузер отправит preflight (OPTIONS)

1 Preflight request (OPTIONS)

Браузер отправляет OPTIONS, чтобы узнать, разрешён ли запрос.

```
OPTIONS /api/orders HTTP/1.1
Origin: http://localhost:5500
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type, authorization
```

2 CORS response

Сервер отвечает CORS headers. Если всё разрешено — браузер выполняет настоящий запрос.

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 600
```

3 Actual request (POST)

После успешного preflight браузер отправляет настоящий запрос.

```
POST /api/orders HTTP/1.1
Origin: http://localhost:5500
Content-Type: application/json
Authorization: Bearer <token>
{ "productId": 1, "quantity": 1 }
```

```
HTTP/1.1 201 Created
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Credentials: true
Content-Type: application/json
{ "id": 123, "status": "created" }
```

Сервер (Backend)
http://localhost:3000

Express.js + cors middleware

```
import express from 'express';
import cors from 'cors';

const app = express();

app.use(cors({
  origin: 'http://localhost:5500',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorication'],
  credentials: true
}));

app.use(express.json());

app.post('/api/orders', (req, res) => {
  // обработка заказа
  res.status(201).json({ id: 123, status: 'created' });
});
```

Сервер разрешает запросы с **http://localhost:5500** и возвращает необходимые CORS headers.

Если **origin** не разрешён, сервер не возвращает **CORS headers**, и браузер блокирует доступ к **response** (даже если сервер вернул 200 OK).

Что такое preflight?
- Браузер автоматически отправляет OPTIONS для проверки, разрешён ли кросс-доменный запрос.
- Используется для «непростых» запросов (методы, заголовки, типы контента).
- Если сервер не разрешает запрос — настоящий запрос не отправляется.

Простые запросы (без preflight)
- GET, HEAD, POST
- Только простые заголовки (например, Content-Type: text/plain, application/x-www-form-urlencoded, multipart/form-data)
- Без Authorization и других кастомных заголовков.

Важно
- CORS — это механизм безопасности браузера.
- Postman, curl и серверные клиенты не использует CORS.
- Access-Control-Max-Age позволяет кэшировать результат preflight.

Как выглядит OPTIONS preflight

```http
OPTIONS /api/orders HTTP/1.1
Origin: http://localhost:5500
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization,content
```

Что важно

- Access-Control-Request-Method сообщает, какой метод планируется.
- Access-Control-Request-Headers перечисляет заголовки будущего запроса.
- Сам запрос POST браузер отправит только после успешной проверки.

Пример успешного ответа на preflight

```http
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Methods: GET,POST
Access-Control-Allow-Headers: Content-Type,Authorizat
```

Что важно

- 204 — успешный ответ без тела.
- Allow-Methods перечисляет разрешённые для CORS методы.
- Allow-Headers разрешает заголовки Content-Type и Authorization.

Почему JSON + Bearer часто вызывают OPTIONS?

- Заголовок Authorization не входит в список CORS-safelisted request headers.
- Content-Type: application/json также не относится к CORS-safelisted Content-Type.
- Поэтому cross-origin POST с JSON и Bearer JWT требует предварительной проверки.
- Это не ошибка: браузер автоматически выполняет необходимую проверку.

Основные CORS-заголовки

Заголовок	Для чего нужен
Origin	Источник страницы, отправляющей запрос
Access-Control-Allow-Origin	Какому origin можно читать ответ
Access-Control-Allow-Methods	Какие методы допустимы при preflight
Access-Control-Allow-Headers	Какие заголовки запроса допустимы
Access-Control-Allow-Credentials	Разрешён ли credentialed CORS
Access-Control-Max-Age	Срок кэширования результата preflight

Учебный вопрос 3. Настройка CORS в Express

- Перейдём от HTTP-заголовков к настройке нашего сервера.

Что такое middleware cors?

- Middleware — функция, выполняющаяся в Express при обработке запросов.
- Пакет cors автоматически формирует необходимые CORS-заголовки ответа.
- Он НЕ проверяет JWT и не запрещает произвольным HTTP-клиентам обращаться к API.
- В нашем обычном варианте, где всё работает на порту 3000, пакет cors не требуется.

Установка пакета cors

```bash
npm install cors
```

Что важно

- Команду выполняют в папке проекта backend.
- Пакет устанавливается только для сценария с разными origins.

Разрешаем frontend на порту 5500

```javascript
import express from "express";
import cors from "cors";

const app = express();
app.use(cors({ origin: "http://localhost:5500" }));
app.use(express.json());
```

Что важно

- Пример для отдельного frontend на localhost:5500 и API на localhost:3000.
- Добавляйте middleware до объявления API-маршрутов.
- Конкретный origin предпочтительнее бесконтрольного разрешения всех сайтов.

Express cors middleware: Origin → policy → Access-Control-Allow-Origin

Internet Shop
Learn. Build. Deploy.

Браузер отправляет заголовок Origin. Express cors сравнивает его с разрешённой политикой.
Если origin разрешён — сервер возвращает соответствующий заголовок, иначе браузер блокирует ответ.

Браузер (Frontend)
http://localhost:5500

http://localhost:5500

Internet Shop (Frontend)

```
fetch("http://localhost:3000/api/products", {
  method: "GET",
  headers: {
    "Origin": "http://localhost:5500"
  }
});
```

Браузер автоматически добавляет заголовок Origin при кросс-доменном запросе.

Заголовок Origin

Содержит источник запроса (scheme + host + port).

Пример: `Origin: http://localhost:5500`
Отправляется автоматически для fetch, XHR и некоторых других запросов.

1. Запрос
Origin: http://localhost:5500

Express приложение (Backend)
http://localhost:3000

Настройка cors middleware

```
import express from 'express';
import cors from 'cors';

const app = express();

const allowedOrigins = [
  'http://localhost:5500',
  'https://shop.example.com'
];

app.use(cors({
  origin: (origin, callback) => {
    if (!origin) return callback(null, true); // например, для Postman
    if (allowedOrigins.includes(origin)) {
      return callback(null, true);
    } else {
      return callback(new Error('Not allowed by CORS'));
    }
  },
  credentials: true
}));
```

Политика (policy)

Разрешённые origins:
- http://localhost:5500
- https://shop.example.com
- https://admin.example.com

Разрешённый origin (успешный запрос)

2. Ответ сервера (CORS headers)

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Credentials: true
Content-Type: application/json

{ "products": [ ... ] }
```

3. Браузер получает ответ и даёт доступ к данным

http://localhost:5500

Данные успешно получены!

Ноутбук	999 $
Смартфон	499 $
Наушники	199 $

Другой origin (заблокированный)

http://evil.com

Злой сайт (evil.com)

```
fetch("http://localhost:3000/api/products")
```

1. Запрос
Origin: http://evil.com

Origin не разрешён политикой

cors middleware не добавляет Access-Control-Allow-Origin для этого origin.

2. Ответ сервера

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "products": [ ... ] }
```

3. Браузер блокирует доступ к response

http://evil.com

Ошибка CORS
Доступ к response заблокирован браузером. JavaScript не может прочитать данные.

Важно помнить
- CORS — это механизм безопасности браузера, а не сервера.
- Сервер всегда может вернуть ответ, но браузер решает, может ли JavaScript получить к нему доступ.
- Postman, curl и серверные клиенты не используют CORS.

Когда разрешать origin
- Используйте белый список (whitelist).
- В продакшене указывайте конкретные origins (без *).
- Для cookie (credentials: true) нельзя использовать *.
- Учитывайте разные окружения (dev, staging, prod).

Чего не делать
- Не использовать `Access-Control-Allow-Origin: *` при `credentials: true`.
- Не разрешать все источники без необходимости.
- Не полагаться на CORS как на единственную защиту API.
- Всегда дополнительно проверяйте аутентификацию и права.

Дополнительная защита
- Проверяйте Authorication / session.
- Используйте CSRF-защиту (для cookie-based auth).
- Настраивайте правильные CORS headers.
- Ограничивайте методы и заголовки.
- Логируйте отклонённые запросы.

Явная политика для методов и заголовков

```javascript
app.use(cors({
  origin: "http://localhost:5500",
  methods: ["GET", "POST", "PATCH", "DELETE"],
  allowedHeaders: ["Content-Type", "Authorization"]
}));
```

Что важно

- methods определяет методы, разрешённые политикой CORS.
- allowedHeaders включает JSON и Bearer JWT.
- При app.use(cors(...)) библиотека обрабатывает предварительные OPTIONS-запросы.

Что означает origin: "*"?

- Звёздочка разрешает чтение ответа браузерным JavaScript с любого origin, если запрос не использует credentials.
- Это может подходить для намеренно публичного API.
- Звёздочка не заменяет проверку JWT, ролей и прав доступа.
- Для credentialed CORS сочетание Access-Control-Allow-Origin: * запрещено.

Нужно ли добавлять CORS в наш проект?

- Если HTML и API отдаёт один Express-сервер через localhost:3000, никакой cross-origin ситуации нет.
- Если frontend вынесен на localhost:5500 или другой домен, настройка CORS становится актуальной.
- Не добавляйте cors «для исправления всего»: сначала сравните scheme, host и port.

Диагностика CORS в браузере

- Откройте F12 → Network и Console.
- Проверьте Origin исходящего запроса и Access-Control-Allow-Origin в ответе.
- Если есть OPTIONS, отдельно изучите его запрос и ответ.
- Различайте CORS-ошибку, сетевую ошибку, 401 Unauthorized и 403 Forbidden.

CORS, 401 и 403 — это разные ситуации

Ситуация	Что означает	Где исправлять
CORS error	Браузер не разрешил JS прочитать ответ	Настройка origins и заголовков
401 Unauthorized	Нет действительной аутентификации	JWT, login, requireAuth
403 Forbidden	Пользователь вошёл, но прав недостаточно	role, requireRole, ownership
Network error	Сервер недоступен или сбой сети	Адрес, порт, сеть, сервер

Учебный вопрос 4. Credentials и cookies

- Разберём второй способ передачи данных входа: cookie вместо Bearer JWT.

Определение: credentials

- Credentials — учётные данные, используемые браузером при HTTP-аутентификации, например cookie с идентификатором сессии.
- Fetch параметр credentials управляет отправкой браузерных credentials в запросах.
- В нашем проекте JWT добавляется вручную в Authorization: Bearer; это не cookie-based authentication.
- credentials: "include" нужен, например, для cross-origin запросов с session cookie.

Пример: запрос с cookie

```javascript
const response = await fetch(
  "http://localhost:3000/api/auth/me",
  { credentials: "include" }
);
```

Что важно

- Пример другой архитектуры: серверная сессия в cookie.
- Включение credentials не гарантирует отправку cookie: действуют также правила SameSite, Secure и политики браузера.
- Это не замена нашего существующего apiFetch() с Bearer JWT.

Express: credentialed CORS

```javascript
app.use(cors({
  origin: "http://localhost:5500",
  credentials: true
}));
```

Что важно

- Для credentialed CORS сервер должен указать конкретный origin.
- credentials: true включает заголовок Access-Control-Allow-Credentials: true.
- origin: "*" вместе с credentialed CORS не допускается.

Credentialed CORS: credentials include + explicit origin + allow credentials

Полный поток кросс-доменных запросов с учётом учетных данных (cookies)

Frontend (Browser)

http://localhost:5500

http://localhost:5500

```
fetch("http://localhost:3000/api/auth/me", {
  credentials: "include"
})
  .then(res => res.json())
  .then(data => console.log(data));
```

Браузер отправляет cookies в кросс-доменных запросах только если указано `credentials: "include"`.

Кросс-доменный запрос

```
GET /api/auth/me
Origin: http://localhost:5500
Cookie: session=abc123...
```

Cookies отправляются в запросе

Express API (Backend)

http://localhost:3000

Express

Настройка CORS (Express)

```
app.use(cors({
   origin: 'http://localhost:5500',
   credentials: true
}));
```

Ответ сервера (заголовки)

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin:
http://localhost:5500
Access-Control-Allow-Credentials: true
```

Origin должен быть указан явно и совпадать с origin фронтенда.

Ответ

(с CORS заголовками)

Браузер получает ответ

http://localhost:5500

http://localhost:5500

```
// Данные доступны JavaScript
{
  "user": {
    "id": 1,
    "email": "user@example.com"
  }
}
```

Доступ разрешён

Ответ доступен JavaScript, можно работать с данными.

* + credentials — нельзя

Неверная конфигурация CORS (не работает с учетными данными):

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

Использовать wildcard (*) вместе с credentials нельзя.

Когда запрос включает учетные данные (cookies), браузер отклоняет ответ, если `Access-Control-Allow-Origin: *`. Необходимо указать конкретный origin (например, `http://localhost:5500`).

Итоги: 3 ключевых условия для credentialed CORS

1. **credentials: "include"**
   Браузер отправляет cookies в кросс-доменных запросах.
2. **Access-Control-Allow-Origin должен быть конкретным**
   Укажите явный origin (например, http://localhost:5500).
3. **Access-Control-Allow-Credentials: true**
   Обязательно разрешите использование учетных данных на бэкенде.

Основные атрибуты cookie

Атрибут	Простое объяснение
HttpOnly	JavaScript не может прочитать cookie через document.cookie
Secure	Cookie передаётся по защищённому соединению (исключения для localhost определяет браузер)
SameSite	Ограничивает отправку cookie в cross-site ситуациях
Path	Задаёт, на какие пути распространяется cookie
Max-Age / Expires	Определяет срок действия cookie

SameSite: три режима

- Strict — наиболее строго ограничивает отправку cookie при переходах с других сайтов.
- Lax — допускает некоторые переходы на сайт верхнего уровня безопасными методами.
- None — допускает cross-site отправку, требует Secure.
- SameSite не равен CORS: это отдельное правило использования cookie.

Bearer JWT и HttpOnly cookie: сравнение

Bearer JWT в нашем проекте	Session cookie (другая архитектура)
JS добавляет Authorization вручную	Браузер может прикладывать cookie автоматически
JWT хранится в authState.js до F5	Cookie может сохраняться между загрузками
Для обычного использования JWT не нужен credentials: include	Для cross-origin cookie fetch обычно нужен include
Нужны защита от XSS и проверка на backend	Нужны защита от XSS, CSRF и проверка на backend

Учебный вопрос 5. CSRF и безопасная конфигурация

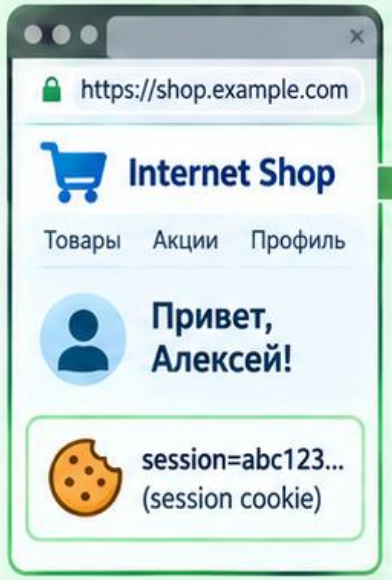
- Разберём угрозу, особенно важную при автоматической отправке cookie.

Определение: CSRF

- CSRF (Cross-Site Request Forgery) — подделка межсайтового запроса.
- Злоумышленник побуждает браузер вошедшего пользователя отправить нежелательный запрос на другой сайт.
- Если браузер автоматически приложит session cookie, сервер может принять запрос как запрос самого пользователя.
- CORS сам по себе не является защитой от CSRF.

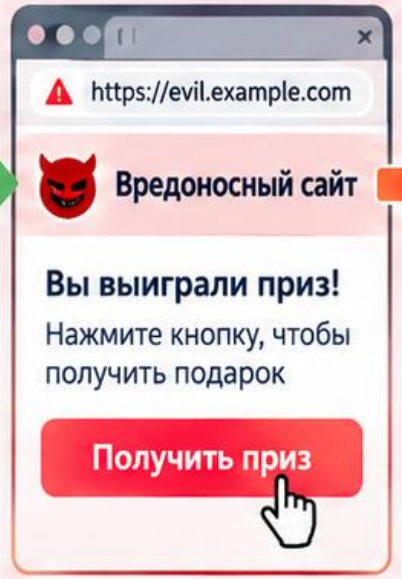
CSRF: вредоносный сайт → браузер с session cookie → защищённое действие

1. Пользователь уже вошёл в Internet Shop



Пользователь уже вошёл в Internet Shop

2. Пользователь открывает вредоносный сайт



Пользователь открывает вредоносный сайт (например, по ссылке в письме, мессенджере и т.п.)

3. Браузер с session cookie



Браузер автоматически добавляет session cookie к запросам на домен Internet Shop

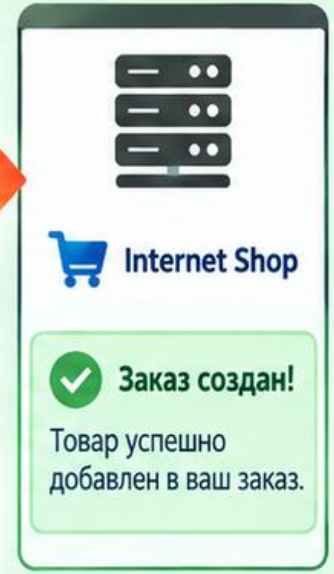
4. state-changing request к Internet Shop



Запрос отправляется на защищённый ресурс Internet Shop

Без дополнительной защиты запрос может быть выполнен от имени пользователя.

5. Защищённое действие



Например:
создание заказа,
изменение email,
добавление адреса и т.п.

Меры защиты

SameSite
ограничивает отправку cookie в cross-site сценариях.

CSRF token
сервер проверяет специальный токен для state-changing запросов.

Origin verification
проверка Origin/Referer для чувствительных запросов.

Правильные HTTP methods
не выполнять изменение состояния через GET; использовать POST/PATCH/DELETE.

Главная идея: cookie-based authentication требует защиты от CSRF.

Пример CSRF простыми словами

- Пользователь вошёл в интернет-магазин и не вышел из него.
- Затем он открывает страницу злоумышленника.
- Страница пытается инициировать действие на сайте магазина.
- Если сервер полагается только на автоматически отправляемую cookie и не проверяет защиту, нежелательное действие может быть выполнено.

Как защищают cookie-based приложение от CSRF

- Не изменяют данные запросами GET.
- Для изменяющих запросов проверяют CSRF-токен и/или Origin/Referer согласно принятой схеме защиты.
- Настраивают подходящий SameSite и используют HTTPS + Secure cookie.
- Эти меры дополняют, а не заменяют аутентификацию и авторизацию.

CORS не заменяет безопасность backend

- Postman, curl и другие HTTP-клиенты не обязаны соблюдать ограничения браузера CORS.
- Даже если пункт «Admin» скрыт в интерфейсе, API должен проверять роль.
- JWT, requireAuth(), requireRole(), проверка владельца объекта и валидация нужны независимо от CORS.
- При наличии cookie-auth дополнительно продумайте CSRF-защиту.

Полный secure cross-origin flow — Internet Shop

От браузера до базы данных и обратно. CORS, аутентификация, авторизация и защита.

Internet Shop
Learn. Build. Deploy.

1 Frontend (Browser) http://localhost:5500

Internet Shop
- Главная
- Каталог
- Мои заказы
- Профиль

Купить товар

```
fetch('http://localhost:3000/api/orders', {
  method: 'POST',
  credentials: 'include',
  headers: { 'Content-Type': 'application/json'},
  body: JSON.stringify({ productId: 1 })
})
```

Пользователь
уже вошёл в систему
(session cookie сохранён)

2 CORS Preflight (OPTIONS)

Браузер отправляет preflight запрос для "непростого" запроса (POST + JSON).

```
OPTIONS /api/orders
Origin: http://localhost:5500
Access-Control-Request-Method: POST
Access-Control-Request-Headers:
content-type, authorization
```

3 Express CORS (проверка origin)

Express.js

cors middleware

Проверяет origin и возвращает CORS headers, если разрешён.

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin:
  http://localhost:5500
Access-Control-Allow-Methods:
  GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers:
  content-type, authorization
Access-Control-Allow-Credentials: true
  Access-Control-Max-Age: 600
```

CORS разрешён
Браузер может отправить настоящий запрос.

4 Actual Request (с cookie)

Браузер отправляет настоящий запрос с **credentials: "include"**. Cookie автоматически добавляется.

```
POST /api/orders
Origin: http://localhost:5500
Cookie: sessionId=abc133
Authorication: Bearer <token>
Content-Type: application/json

{ "productId":1, "quantity": 1 }
```

5 Auth Middleware (requireAuth / requireRole)

requireAuth()
(проверка сессии/JWT)

requireRole('user')
(проверка роли)

1 401 Unauthorized
Токен отсутствует, недействителен или сессия истекла.

1 403 Forbidden
Пользователь аутентифицирован, но нет прав.

6 Controller → Service

OrdersController
(обработка запроса)

OrdersService
(бизнес-логика)

PostgreSQL
(база данных)

Создание заказа в базе данных

```
INSERT INTO orders ...
```

7 Response (CORS headers)

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin:
  http://localhost:5500
Access-Control-Allow-Credentials:
  true
Content-Type: application/json

{
  "id": 101,
  "status": "created",
  "message": "Order created"
}
```

8 Browser → UI

Браузер получает ответ, проверяет CORS headers и передаёт данные в JavaScript.

Internet Shop

Заказ #101 успешно создан!

Перейти к заказам

Данные отображаются на странице.

CORS Denied (origin не разрешён)

Если origin не в whitelist, сервер не добавляет Access-Control-Allow-Origin, и браузер блокирует ответ.

```
OPTIONS /api/orders
Origin: http://evil.com
```

→ `HTTP/1.1 403 Forbidden` (без CORS headers)

→ **Браузер:** CORS error — Доступ к response заблокирован. JavaScript не может прочитать данные.

1 401 Unauthorized

```
HTTP/1.1 401 Unauthorized
Access-Control-Allow-Origin:
  http://localhost:5500
Access-Control-Allow-Credentials: true
Content-Type: application/json

{ "error": "Unauthorized" }
```

1 403 Forbidden

```
HTTP/1.1 403 Forbidden
Access-Control-Allow-Origin:
  http://localhost:5500
Access-Control-Allow-Credentials: true
Content-Type: application/json

{ "error": "Forbidden" }
```

Коды ответов

200 OK	Запрос выполнен успешно
1 401 Unauthorized	Нет действующей аутентификации
1 403 Forbidden	Нет прав для выполнения действия
CORS Denied	Origin не разрешён (браузер блокирует)

Cookie и CORS

- fetch(..., { credentials: "include"})
- На сервере: Access-Control-Allow-Credentials: true
- Access-Control-Allow-Origin: конкретный origin (не *)
- Cookie: HttpOnly, Secure, SameSite (Lax/Strict)

Защита от CSRF

- Используйте SameSite=Lax/Strict для session cookies
- Для state-changing запросов — CSRF токен (double submit или synchronizer token)
- Проверяйте заголовок Origin/Referer
- Используйте правильные HTTP методы (POST/PUT/DELETE), не GET

Дополнительные рекомендации

- Настраивайте CORS для конкретных origin (whitelist)
- Ограничивайте разрешённые методы и заголовки
- Логируйте отклонённые запросы
- Используйте короткий срок жизни токенов
- Проверуйте роли на уровне middleware

Результат

Безопасное HTTP-взаимодействие между frontend и backend с учётом CORS, аутентификации, авторизации и защиты от CSRF.

Практика: два сценария

Сценарий	Какой результат ожидается
Обычный проект: frontend :3000 + API :3000	CORS не нужен; обычный fetch("/api/products")
Учебный CORS: frontend :5500 + API :3000	Проверяем Origin и настройку cors
Вариант с POST JSON + Authorization	При cross-origin запросе браузер отправляет OPTIONS preflight
Отдельный cookie-based пример	Потребуются credentials: include и разрешение на сервере

Порядок проверки в DevTools

- 1. Откройте frontend на localhost:5500 только для учебной демонстрации CORS.
- 2. Отправьте запрос на API localhost:3000.
- 3. В Network найдите Origin и, при наличии, запрос OPTIONS.
- 4. Сравните ответ до и после добавления cors middleware.
- 5. Вернитесь к frontend на localhost:3000: объясните, почему CORS больше не требуется.

Контрольные вопросы

1. Из каких трёх частей состоит origin?
2. Почему localhost:5500 и localhost:3000 считаются разными origins?
3. Что ограничивает Same-Origin Policy?
4. Что означает CORS и какие заголовки участвуют в проверке?
5. Для чего браузер отправляет OPTIONS preflight?
6. Почему JSON с Authorization часто вызывает preflight?
7. Нужен ли CORS, когда frontend и API работают на localhost:3000?
8. Почему CORS нельзя считать защитой API от Postman?
9. Для чего используют credentials: include, HttpOnly и SameSite?
10. Что такое CSRF и почему он важен для cookie-based auth?

Итоги лекции

1. Origin = scheme + host + port; путь URL в origin не входит.
2. Same-Origin Policy ограничивает чтение чужих ответов браузерным JavaScript.
3. CORS позволяет серверу явно разрешить доступ из других origins.
4. Preflight OPTIONS — предварительная проверка методов и заголовков.
5. Express cors middleware формирует CORS-заголовки, но не выполняет авторизацию.
6. Наш Internet Shop на localhost:3000 не нуждается в CORS при same-origin.
7. Credentialed CORS с cookie требует конкретного origin и корректных настроек.
8. CSRF особенно важен для cookie-based аутентификации; backend всегда проверяет права.